

A Voxel-Spacing-Aware Extension of PyRadiomics for Anisotropic Texture Analysis

Technical implementation audit generated from local source comparison

September 10, 2026

Abstract

Radiomic texture features are frequently extracted from anisotropic CT and MRI acquisitions. Standard PyRadiomics texture computation enumerates voxel relationships in discrete index space, where a one-voxel offset along the slice direction is treated analogously to a one-voxel offset in-plane. When voxel spacing is anisotropic, these offsets can correspond to substantially different physical distances. A common workaround is isotropic resampling, but resampling couples geometric correction to interpolation-induced signal modification. This manuscript describes and audits a modified PyRadiomics implementation designed to preserve the gray levels supplied to texture computation while incorporating physical voxel spacing into selected texture operations. The central methodological idea is to separate geometric correction from interpolation-induced signal modification. GLCM receives voxel-spacing-aware feature-level angular aggregation; NGTDM receives a distance-weighted neighborhood mean; GLRLM, GLDM and GLSZM are computed after a finite-volume zero-order-hold representation of anisotropic voxel support. The implementation is opt-in, preserves standard PyRadiomics behavior when disabled, and is accompanied by synthetic validation of rounding error, computational expansion and comparison against isotropic resampling baselines.

1 Introduction

Medical images often have anisotropic voxel spacing, especially in clinical CT and MRI where through-plane spacing can differ markedly from in-plane spacing. Radiomic features extracted from such images may therefore conflate image texture, acquisition geometry, and preprocessing choices. PyRadiomics is a widely used radiomics library, but many texture matrices are computed using discrete offsets defined in index space. This is computationally natural, yet geometrically incomplete when a displacement of one voxel does not represent the same physical length along all axes.

2 Motivation: anisotropy, interpolation and index-space texture bias

Let voxel spacing be $s = (\Delta_z, \Delta_y, \Delta_x)$. A discrete offset vector $a = (a_z, a_y, a_x)$ corresponds to the physical offset length

$$d_a = \sqrt{(a_z \Delta_z)^2 + (a_y \Delta_y)^2 + (a_x \Delta_x)^2}. \quad (1)$$

In an anisotropic volume, offsets such as $(1, 0, 0)$ and $(0, 1, 0)$ are both one index step but may be physically very different. Isotropic resampling addresses this geometric inconsistency by creating an

isotropic grid, but it also interpolates intensities and modifies local spatial-frequency content. The modified implementation studied here follows a different design: preserve the gray levels supplied to texture computation and incorporate spacing into the interpretation, aggregation or finite-volume representation of voxel relationships.

3 Overview of standard PyRadiomics texture computation

Standard PyRadiomics builds texture matrices by enumerating neighbors or runs using discrete offsets derived from distance and dimensionality settings. Directional matrices may then be averaged or weighted according to existing weighting norms. Shape features already use physical spacing. First-order features primarily summarize intensity distributions and are less directly tied to neighborhood geometry, except for volume-scaled quantities such as Total Energy.

4 Design principles of voxel-spacing-aware extraction

The proposed design follows four principles. First, the gray levels supplied to texture computation are preserved. Second, no isotropic interpolation is introduced. Third, physical voxel spacing is used only where the mathematical structure of the descriptor admits a meaningful correction or finite-volume representation. Fourth, the new behavior is opt-in and must not alter standard PyRadiomics behavior when disabled.

For a set of non-zero offsets A , let the index-space offset length be

$$\ell_a = \sqrt{a_z^2 + a_y^2 + a_x^2}, \quad (2)$$

and let the anisotropy-relative physical stretch be

$$r_a = \frac{d_a}{\ell_a}. \quad (3)$$

Valid texture offsets exclude the zero vector and voxel spacings are strictly positive; therefore $r_a > 0$. The conceptual inverse-distance directional weights are

$$w_a = \frac{1}{r_a}, \quad W_a = \frac{w_a}{\sum_{b \in A} w_b}. \quad (4)$$

In the implementation, GLCM uses $1/\max(r_a, 10^{-5})$ as a numerical floor. NGTDM computes $r_a = d_a/(\ell_a + 10^{-12})$ in the C backend and uses $1/(r_a + 10^{-6})$ for the local weighted mean. These constants are numerical safeguards only; they are negligible relative to valid non-zero offset stretches in the present extraction settings. These weights are not a claim of continuous-space radiomics. They are a discrete-grid correction for physically unequal offsets. For matrix families handled by finite-volume representation, anisotropic voxels are instead expanded by zero-order hold onto an isotropic subcell lattice; this replicates native gray levels and does not interpolate them.

5 Software architecture and implementation

The modified package adds frontend Python logic, a shared spacing utility, and C wrapper/backend functions. In the Python layer, feature classes inspect settings such as `weightingNorm='voxelSpacing'` or receive `voxelSpacing`. In the wrapper layer, `calculate_ngtdm_spacing` passes spacing arrays

into C for NGTDM. Finite-volume GLRLM, GLDM and GLSZM are prepared in Python by repeat-factor calculation and zero-order-hold expansion before standard matrix construction. In the back-end, physical distances are computed from discrete offsets and spacing where required. In the current upstream-overlay audit, `git diff --stat` reports 24 tracked files changed; `git status` additionally lists new files such as `spacing_utils.py`, `vnn_preprocessing.py`, the voxel-spacing synthetic test, and example parameter files. Table 1 therefore lists the core implementation files relevant to the method rather than serving as an exhaustive file-count claim. Figure 2 summarizes this flow.

All validation experiments reported here use segment-based, fully three-dimensional texture extraction with `force2D=False`; they are not two-dimensional or 2.5-dimensional texture calculations. Internally, SimpleITK images are converted to NumPy arrays in (z, y, x) order, whereas SimpleITK reports spacing in (x, y, z) order. The voxel-spacing-aware paths therefore use `GetSpacing()[:-1]` before applying spacing to array-index offsets or finite-volume repeat factors. Image direction/orientation matrices are not incorporated into the voxel-spacing-aware calculations; only voxel spacing magnitudes are used.

6 Feature-family-specific modifications

6.1 GLCM

The GLCM modification is conservative. Co-occurrence pairs are still enumerated in native index space, producing directional matrices $P_a(i, j)$. The implementation does not first combine these matrices into a single weighted matrix. Instead, each GLCM feature f is computed per direction and the final angular aggregation is weighted:

$$F_f = \sum_{a \in A} W_a f(P_a). \quad (5)$$

This distinction matters because, for non-linear GLCM features, $\sum_a W_a f(P_a) \neq f(\sum_a W_a P_a)$. The modification should therefore be described as voxel-spacing-aware feature-level angular aggregation, not as weighted matrix fusion and not as a full physical-space GLCM.

6.2 GLRLM

GLRLM is computed after finite-volume zero-order-hold expansion when `weightingNorm='voxelSpacing'` is enabled in anisotropic images. Let $h = \min(\Delta_z, \Delta_y, \Delta_x)$ and

$$q_k = \text{round}\left(\frac{\Delta_k}{h}\right), \quad k \in \{z, y, x\}. \quad (6)$$

The native binned image is expanded by repeating each voxel q_k times along axis k , creating $\tilde{I} = \text{ZOH}(I; q_z, q_y, q_x)$. The represented spacing and relative spacing error are therefore

$$\Delta_k^* = q_k h, \quad e_k = \frac{\Delta_k^* - \Delta_k}{\Delta_k}. \quad (7)$$

Standard GLRLM construction is then applied to $\tilde{I}$. Run length remains an integer count, but it counts isotropic physical subcells rather than anisotropic native voxels. This integer representation is the main mathematical approximation of the finite-volume method: for example, 2.5:1:1 is represented as 2:1:1, giving $e_z = -20.0\%$, whereas 3.5:1:1 is represented as 4:1:1, giving $e_z = +14.3\%$.

The implementation guards the product $q_z q_y q_x$ through `maximumExpansionFactor`; the finite-volume paths $\log q_k$, Δ_k^* , e_k , and expanded shape, and emit a warning containing R when the expansion factor exceeds 8.

6.3 NGTDM

NGTDM is a stronger descriptor-level extension because the descriptor depends explicitly on a local neighborhood mean. The modified backend preserves native discrete neighborhood topology but replaces the unweighted local mean by

$$\bar{I}(x) = \frac{\sum_{a \in N(x)} I(x+a) w_a}{\sum_{a \in N(x)} w_a}. \quad (8)$$

The usual gray-level accumulation then uses $|I(x) - \bar{I}(x)|$. This is conceptually strong because neighborhood averaging naturally admits inverse-distance physical weighting.

6.4 GLDM

GLDM is also computed on the finite-volume zero-order-hold representation $\tilde{I}$. Standard dependence counting is then applied without fractional dependence weights. Dependence size remains an integer count, avoiding the earlier weighted-dependence prototype in which a continuous value had to be rounded back to a matrix axis.

6.5 GLSZM

GLSZM is topology-based: it describes connected zones of equal gray level. Under anisotropic `voxelSpacing`, the standard connected-component algorithm is applied to the finite-volume zero-order-hold representation $\tilde{I}$. Zone size remains an integer count, but it counts isotropic subcells rather than anisotropic native voxels. This is preferable to post-hoc volume scaling because matrix construction and feature formulas remain standard after representation.

6.6 Shape and first-order considerations

Shape features already use image spacing in standard PyRadiomics. First-order features are mostly intensity-distribution descriptors. The observed Total Energy branch is ancillary and should not be grouped with texture-neighborhood corrections.

6.7 Finite-volume expansion safeguards

For finite-volume families, the computational expansion factor is

$$R = q_z q_y q_x, \quad (9)$$

and the expanded lattice size is

$$N_{\text{expanded}} = (N_z q_z)(N_y q_y)(N_x q_x). \quad (10)$$

The current implementation computes the expanded shape before allocating the expanded image and mask. The shared helper exposes a `maximumExpansionFactor` guard on R , with default value 1000; the finite-volume feature classes call this helper with its default value. The implementation records represented-spacing metadata in logger messages and emits warnings when $\max_k |e_k| > 10\%$

or $R > 8$. It does not currently enforce an absolute N_{expanded} memory guard at library level. For routine use, we recommend also reporting N_{expanded} to the user before allocation and requiring explicit user confirmation or a hard stop for very large expansions such as $R > 16$ or user-defined memory limits.

7 Mathematical formulation

The common mathematical object is the physical offset length d_a . For GLCM, d_a changes the feature-level angular aggregation. For NGTDM, d_a changes the neighborhood mean. For GLRLM, GLDM and GLSZM, anisotropic voxel support is represented by finite-volume zero-order-hold expansion before standard matrix construction.

8 Compatibility with IBSI/PyRadiomics workflows

Backward compatibility depends on default-off behavior. When the new mode is disabled, calls should route to the standard PyRadiomics implementation. The modified NGTDM and finite-volume matrix families should not be claimed as IBSI-compliant replacements; they are opt-in extensions. GLCM can be documented as optional anisotropy-aware feature-level angular aggregation.

9 Additional software validation

Three supplementary validation experiments quantify the main implementation risks: finite-volume rounding, computational expansion and non-equivalence to resampling. The validation tables and figures are distributed with the experiment outputs, including per-feature CSV tables for resampling comparisons.

9.1 Finite-volume rounding sensitivity

The finite-volume representation was tested on spacing ratios 1:1:1, 2:1:1, 2.5:1:1, 3:1:1, 3.5:1:1, 5:1:1 and 5.5:1:1. The validation output records q_k , Δ_k^* , e_k and R . Integer ratios had zero represented-spacing error. The two most important non-integer examples were 2.5:1:1 $\rightarrow$ 2:1:1, with $e_z = -20.0\%$, and 3.5:1:1 $\rightarrow$ 4:1:1, with $e_z = +14.3\%$. Across adjacent spacing-ratio comparisons, the largest family-level median relative feature difference was 0.167 for GLSZM and the lowest Spearman correlation was 0.900 for NGTDM. These results do not remove the integer-rounding limitation, but they quantify it and make the approximation auditable. The adjacent integer comparisons also provide a low-cost floor/ceil sensitivity check around non-integer ratios, supporting round as the minimum absolute represented-spacing error choice under the current integer-repeat model.

9.2 Computational profiling and stress testing

Runtime and memory profiling used three synthetic volume sizes: $16 \times 64 \times 64$, $32 \times 96 \times 96$ and $48 \times 128 \times 128$. The corresponding native ROI sizes were 12,126, 50,328 and 139,028 voxels. Tested expansion factors were $R = 1, 2, 3, 5$ and an adverse axis case with $R = 7$. The most demanding completed case used a $48 \times 128 \times 128$ input with spacing 0.7:0.7:5.0, producing $q = (1, 1, 7)$, an expanded lattice of $48 \times 128 \times 896$, 973,196 ROI voxels after expansion, a runtime of 1.74 s and a

peak memory estimate of 90.55 MB. Median peak memory increased from 2.83 MB at $R = 1$ to 13.56 MB at $R = 7$, while maximum observed runtime remained below 1.75 s in the tested synthetic setting. These data support a moderate-cost claim only for the tested sizes and expansion factors; larger clinical volumes should use the pre-allocation N_{expanded} estimate and configured limits.

9.3 Comparison with isotropic resampling

The resampling baseline compared native extraction, nearest-neighbor resampling, linear resampling, B-spline resampling and voxel-spacing-aware extraction on the same synthetic image family. The per-feature supplementary table contains one row per feature with native value, comparison value, absolute difference, relative difference and method columns for nearest, linear, B-spline and spacing-aware extraction. Relative to native extraction, nearest-neighbor resampling changed 192/225 features (85.3%), linear and B-spline resampling changed 225/225 features (100%), and voxel-spacing-aware extraction changed 192/225 features (85.3%). The median of family-level median relative differences was 0.333 for nearest-neighbor resampling, 0.304 for linear resampling, 0.302 for B-spline resampling and 0.307 for voxel-spacing-aware extraction. Minimum method-level Spearman correlations were 0.900 for nearest-neighbor and voxel-spacing-aware extraction and 0.800 for linear and B-spline resampling. Thus, the spacing-aware method is not equivalent to resampling: it yields differences of comparable magnitude while preserving native gray levels rather than creating interpolated intensities. Figure 10 summarizes the family-level distributions.

10 Limitations and known implementation risks

The principal mathematical limitation is finite-volume integer rounding. When spacing ratios are non-integer, the represented spacing Δ_k^* can differ from the acquired spacing Δ_k , and this error should be reported with the extracted features, logs or validation metadata. The second limitation is memory scaling: zero-order-hold expansion preserves native gray values but increases array size by R . The third limitation is scope: the validation experiments are synthetic and designed to test software behavior, not to establish clinical superiority. The modified NGTDM and finite-volume matrix families should therefore be described as opt-in PyRadiomics extensions rather than IBSI-compliant replacements.

11 Recommendations for upstream PyRadiomics integration

An upstream pull request should be staged around opt-in behavior and backward compatibility. GLCM can be proposed as the most conservative feature-level angular-aggregation correction. NGTDM can be proposed as a well-motivated descriptor extension. GLRLM, GLDM and GLSZM can be proposed as finite-volume zero-order-hold matrix extensions, with explicit documentation that native gray levels are replicated rather than interpolated.

12 Discussion

The main contribution is methodological rather than benchmark-oriented: voxel-spacing-aware extraction separates physical geometry from interpolation-induced signal modification. This matters because isotropic resampling can improve apparent stability by smoothing or regularizing local image structure, but such smoothing changes the signal being measured. The modified implementation instead preserves the gray levels supplied to texture computation and either weights native-grid

relationships or evaluates standard matrices on a replicated finite-volume lattice, depending on the feature family.

The validation results also clarify what the method is not. It is not a continuous-space radiomics formulation, because finite-volume families still require integer repeat factors. GLCM is not a weighted-matrix-fusion method: it preserves directional matrices, evaluates each feature per direction, and changes only the final angular aggregation. It is not equivalent to resampling, because it does not introduce interpolated gray levels and its feature differences follow family-specific patterns. Finally, it is not cost-free: the expanded lattice size must be estimated before allocation and limited in routine use.

13 Conclusion

Voxel-spacing-aware PyRadiomics extensions can address a real geometric limitation of index-space texture computation in anisotropic CT and MRI while preserving the gray levels supplied to texture computation. The most defensible implementation path is selective and opt-in: feature-level angular aggregation for GLCM, distance-weighted local averaging for NGTDM, and finite-volume zero-order-hold representations for GLRLM, GLDM and GLSZM. The principal caveats are integer spacing-ratio rounding, expansion-factor memory scaling and non-IBSI extension status. Reporting Δ_k^* , e_k , R and N_{expanded} , together with visible resampling comparisons, turns these caveats into auditable design constraints.

References placeholder

No internet lookup was performed during generation. See `references_to_search.md` for recommended references to add manually.

Separating geometric correction from interpolation-induced signal modification



Figure 1: Conceptual separation between isotropic resampling and voxel-spacing-aware extraction.

Software architecture for optional voxel-spacing-aware extraction



Default-off behavior should route to standard PyRadiomics; spacing-aware mode passes physical spacing only where needed.

Figure 2: Software architecture of the modified PyRadiomics implementation.

Feature-family taxonomy



Figure 3: Feature-family taxonomy for spacing-aware modifications.

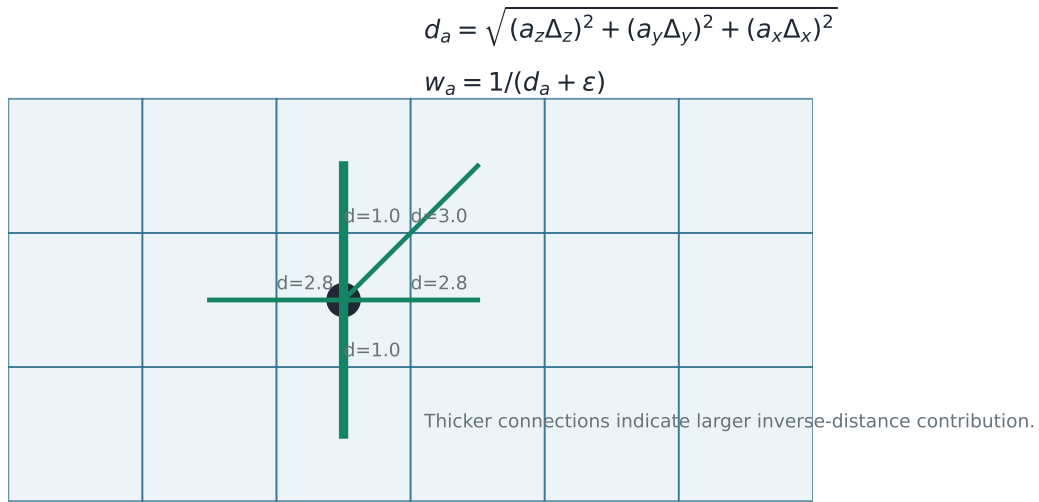


Figure 4: Physical offset weighting in an anisotropic voxel grid.

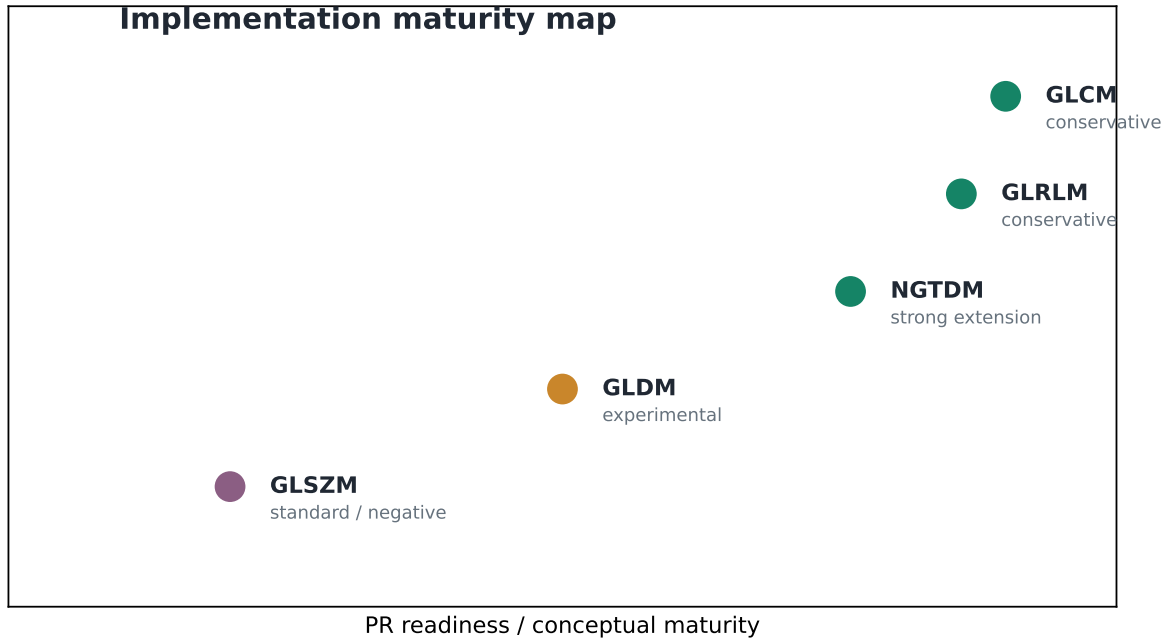


Figure 5: Implementation maturity map for a potential upstream pull request.

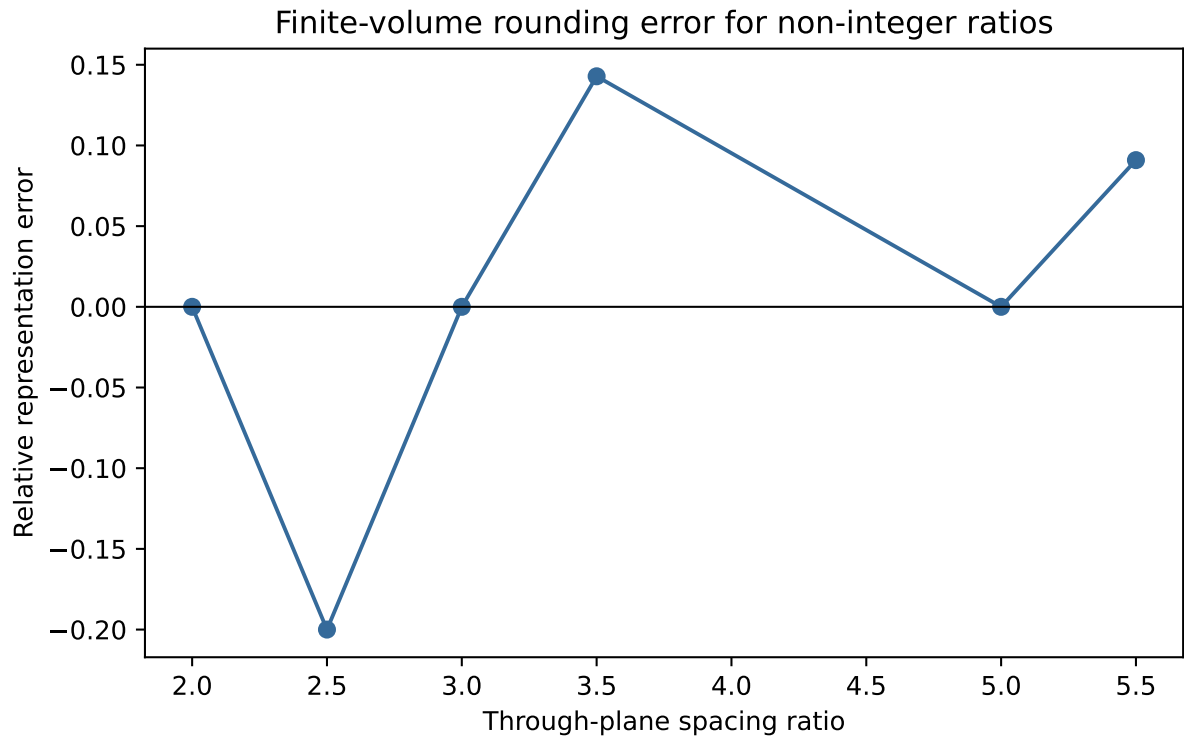


Figure 6: Represented-spacing error introduced by integer finite-volume repeat factors across tested spacing ratios.

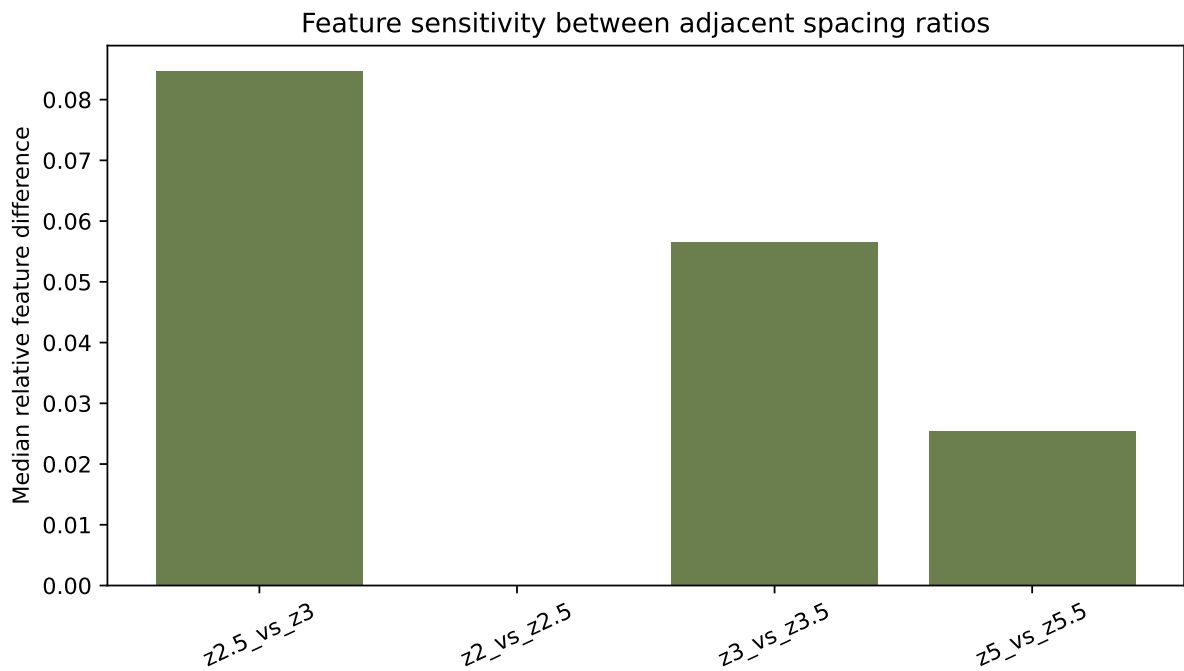


Figure 7: Family-level feature sensitivity across adjacent integer and non-integer finite-volume spacing ratios.

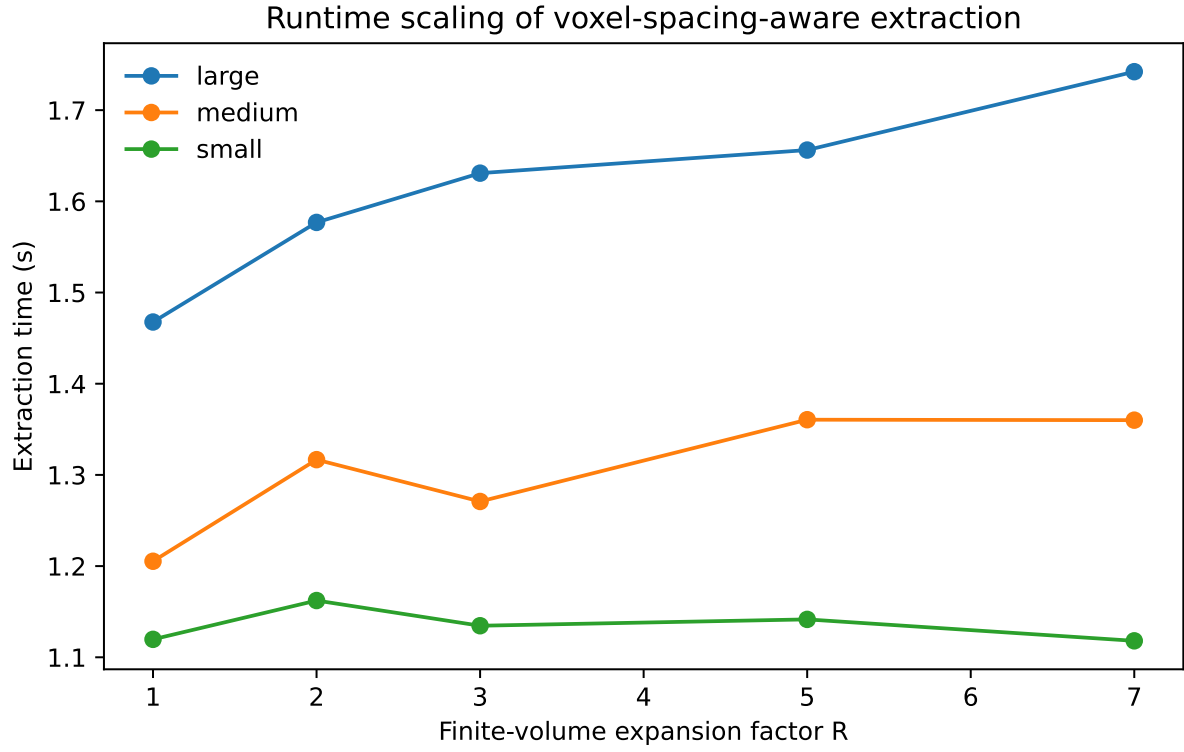


Figure 8: Runtime profile as a function of finite-volume expansion factor in the synthetic stress test.

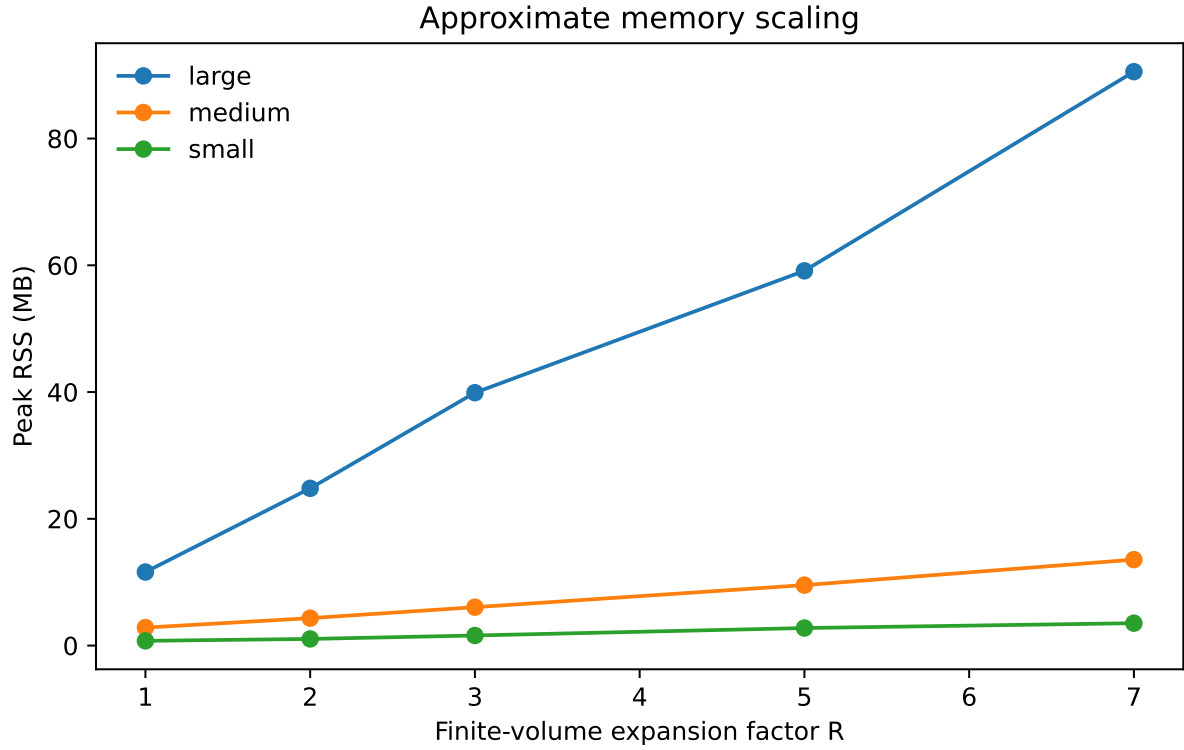


Figure 9: Peak memory profile as a function of finite-volume expansion factor in the synthetic stress test.

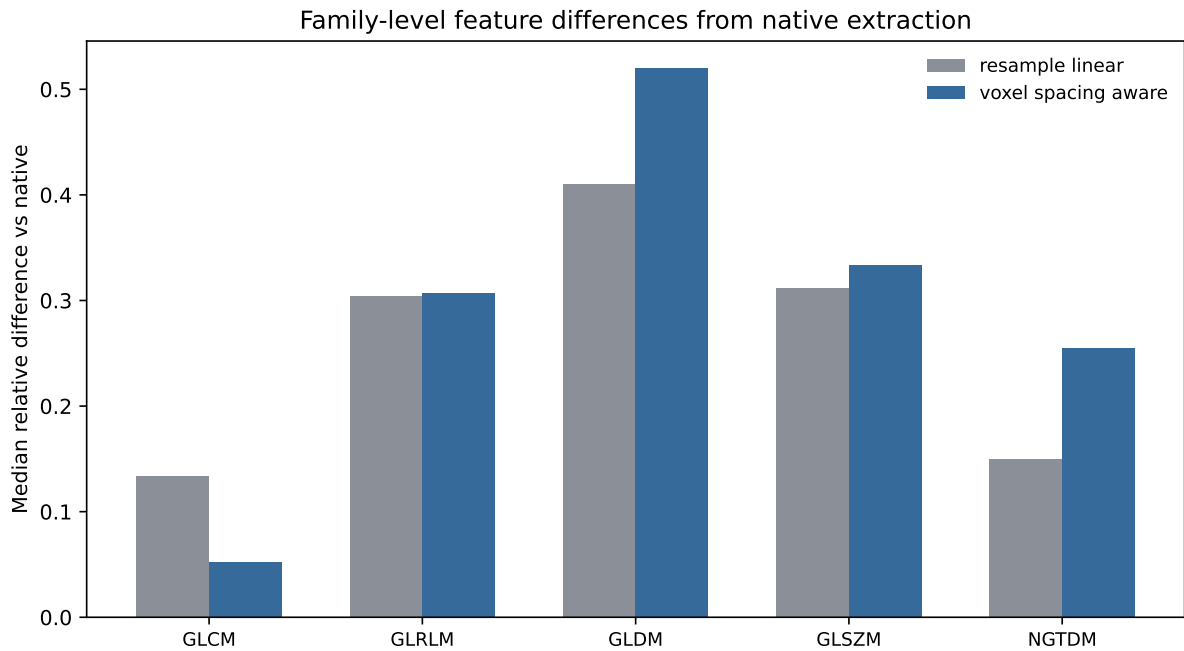


Figure 10: Family-level relative differences from native extraction for nearest-neighbor, linear, B-spline and voxel-spacing-aware extraction.

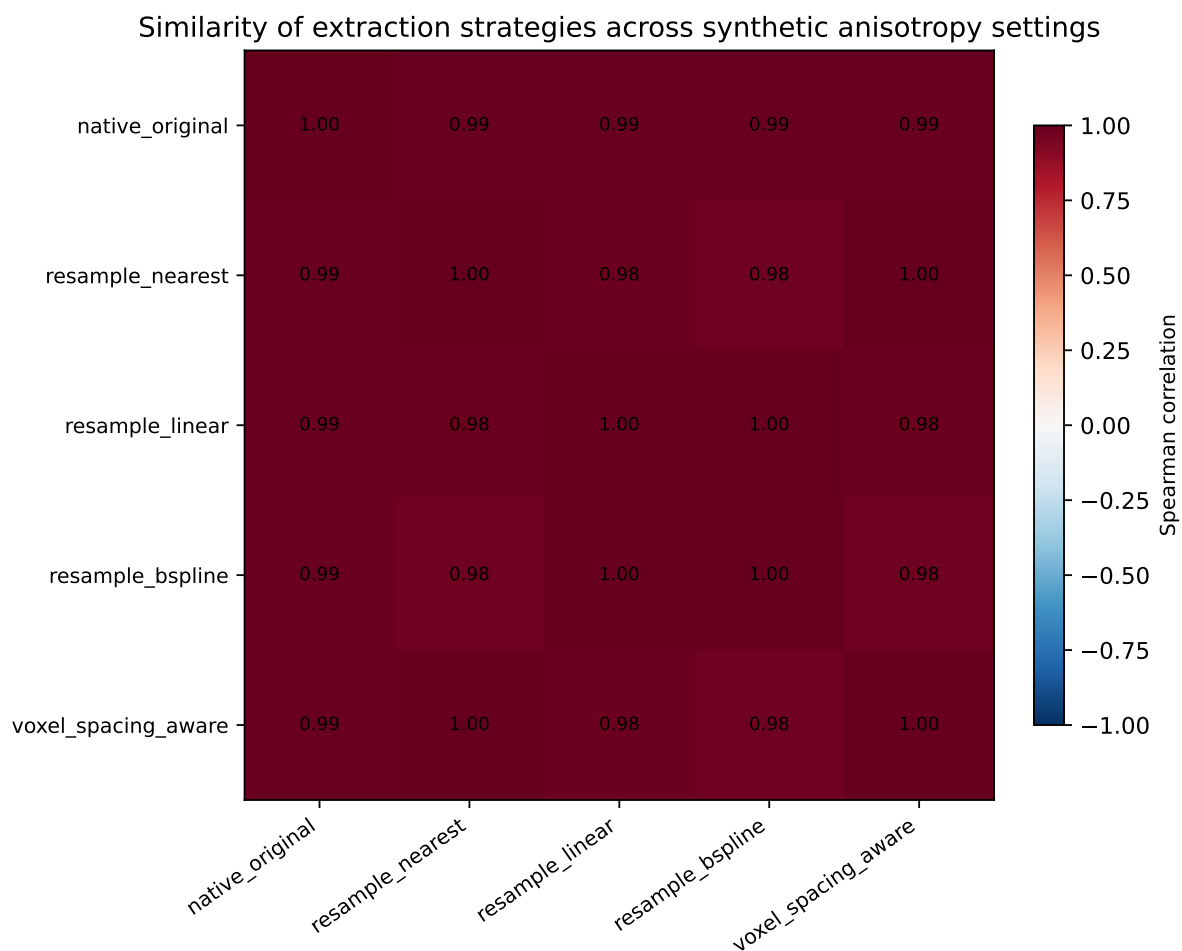


Figure 11: Method similarity across native extraction, isotropic resampling baselines and voxel-spacing-aware extraction.

file	affected layer	family or	new parameters or terms	mathematical meaning	pr maturity
base.py	Frontend/configuration		none detected	Configuration or utility support for optional spacing-aware behavior.	supporting
featureextractor.py	Frontend/configuration		weightingNorm	Configuration or utility support for optional spacing-aware behavior.	supporting
firstorder.py	First-order		voxelSpacing, weightingNorm	Volume-aware branch for Total Energy; most first-order features are not neighborhood geometry descriptors.	ancillary
glcm.py	GLCM		calculate_glcm, compute_offset_distance, spacing_utils, voxelSpacing, weightingNorm	Voxel-spacing-aware feature-level angular aggregation; co-occurrence enumeration remains native index-space.	conservative mature
gldm.py	GLDM		calculate_gldm, spacing_utils, voxelSpacing, weightingNorm	Finite-volume zero-order-hold GLDM construction; dependence size remains an integer matrix axis.	finite-volume mature
glrlm.py	GLRLM		calculate_glrlm, spacing_utils, voxelSpacing, weightingNorm	Finite-volume zero-order-hold GLRLM construction; run length remains an integer matrix axis.	finite-volume mature
glszm.py	GLSZM		calculate_glszm, spacing_utils, voxelSpacing, weightingNorm	Finite-volume zero-order-hold GLSZM construction; zone size remains an integer matrix axis.	finite-volume mature
ngtdm.py	NGTDM		calculate_ngtdm, calculate_ngtdm_spacing, voxelSpacing, weightingNorm	Distance-weighted neighborhood averaging before NGTDM gray-level accumulation.	strong descriptor extension
schemas/schemaFuncs.py	Frontend/configuration		voxelSpacing	Configuration or utility support for optional spacing-aware behavior.	supporting
spacing_utils.py	Shared spacing utility		compute_offset_distance, spacing_utils	Configuration or utility support for optional spacing-aware behavior.	supporting
src/_cmatrices.c	C matrix backend/wrapper	back-	calculate_glcm, calculate_gldm, calculate_glrlm, calculate_glszm, calculate_ngtdm, calculate_ngtdm_spacing	C wrapper/backend support for standard matrix construction and spacing-aware NGTDM.	backend support
src/cmtrices.c	C matrix backend/wrapper	back-	calculate_glcm, calculate_gldm, calculate_glrlm, calculate_glszm, calculate_ngtdm, calculate_ngtdm_spacing	C backend support for standard ma-	backend support

feature family	standard mathematical dependency	mathematical dependency	anisotropy problem	problem	modification type	base definition changes	pr maturity
GLCM	Directional co-occurrence offsets		One index step can represent unequal physical distances		Voxel-spacing-aware feature-level angular aggregation	No	Conservative mature
GLRLM	Directional runs counted in voxels		Native voxel counts underrepresent thick-slice physical support		Finite-volume zero-order-hold matrix construction	Yes opt-in	Finite-volume mature
NGTDM	Local neighborhood mean		Unweighted mean treats near and far anisotropic neighbors equally		Distance-weighted neighborhood averaging	Yes	Strong descriptor extension
GLDM	Dependence size as integer neighbor count		Dependent-neighbor counts are tied to anisotropic native voxel support		Finite-volume zero-order-hold matrix construction	Yes opt-in	Finite-volume mature
GLSZM	Connected zones/topology		Zone size in native voxels differs from physical zone support		Finite-volume zero-order-hold matrix construction	Yes opt-in	Finite-volume mature
First-order	Intensity histogram and scalar intensity moments	histogram and intensity	Mostly not neighborhood-direction dependent; total energy is volume-scaled		Ancillary voxel-volume handling only	Mostly no	Ancillary
Shape	Mesh/voxel geometry descriptors	de-	Standard PyRadiomics already uses spacing for shape geometry		No new texture-mode correction	No	Standard

Table 2: Feature-family classification for voxel-spacing-aware extraction.

family	equation	interpretation
GLCM	$F_f = \sum_a W_a f(P_a)$	Directional co-occurrences are enumerated on the native grid; each feature is computed per direction before weighted angular aggregation.
GLRLM	$R = \text{GLRLM}(\tilde{I})$	Runs remain integer counts on the finite-volume lattice.
NGTDM	$\bar{I}(x) = \frac{\sum_{a \in N(x)} I(x+a) w_a}{\sum_{a \in N(x)} w_a}$	Neighborhood mean is a true descriptor-level distance-weighted average.
GLDM	$G = \text{GLDM}(\tilde{I})$	Dependence size remains an integer count on the finite-volume lattice.
GLSZM	$Z = \text{GLSZM}(\tilde{I})$	Zone size remains an integer count on the finite-volume lattice.

Table 3: Mathematical summary of supported or considered spacing-aware operations.

topic	implication	suggested wording
Backward compatibility	Preserve original behavior unless the opt-in spacing-aware mode is enabled.	Use explicit parameter naming and default-off behavior.
IBSI implications	Modified NGTDM and finite-volume matrix families are not standard IBSI definitions.	Label as extensions rather than replacements.
Semantic risk	Finite-volume expansion changes the matrix construction domain.	Document zero-order-hold representation and expansion factors.
Documentation wording	Avoid claims of continuous-space radiomics.	Use native gray levels preserved, physical offset weighting and finite-volume representation.

Table 4: Compatibility and limitations.

Table 5: Additional software-validation summary.

Experiment	Main comparison	Key metric	Main result	Interpretation
Resampling base-line comparison	Voxel-spacing-aware extraction vs native and isotropic resampling	Median relative difference; correlation	VS median relative difference vs native 0.307; linear resampling 0.304.	VS behaves as a distinct interpolation-free alternative.
Computational profiling	Spacing ratio and ROI size	Runtime vs expansion factor	Observed up to R=7; maximum completed runtime 1.74 s.	Cost scales with finite-volume expansion; warnings are advisable for high R.
Rounding sensitivity	Integer vs non-integer spacing ratios	Relative represented-spacing error	Maximum tested relative z-spacing error 0.2.	Non-integer ratios are approximated by integer repeat factors.
Binning sensitivity	binWidth 5, 10, 25, 50	Median relative difference	VS vs native median relative difference range 0.145-0.233.	Discretization interacts with feature-level differences and should be reported.

checklist item	specific action	priority
Frontend configuration	Keep parameter access explicit and document spacing source.	Required
Wrapper interface	Stabilize PyArg_ParseTuple signatures and optional arguments.	Required
Computational safeguards	Keep configurable expansion-factor limits and report N_{expanded} before allocation.	Required
Docstrings	Document voxelSpacing/weightingNorm semantics per feature class.	Required
Tests	Add isotropic equivalence, anisotropic known-offset, backward compatibility and no-crash tests.	Required
Examples	Provide anisotropic CT/MRI example and compare NR/RS/VS behavior.	Recommended
Parameter naming	Separate distance metric from weighting function.	Recommended
Documentation	Add rounding-error, expansion-size and IBSI-extension language.	Required

Table 6: Pull request readiness checklist.